

A Project Stage-I Report on
TRUSTRAG: A MODULAR FRAMEWORK FOR DETECTING
AND MITIGATING HALLUCINATIONS IN RETRIEVAL-
AUGMENTED GENERATION SYSTEMS

Submitted in Partial fulfillment of requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

By

P. VARUN SAI	23BD1A056A
S. SRISHANTH	23BD1A0572
S. AAKASH	23BD1A0573
V. MAITHRESH	23BD1A057D

Under the guidance of

Mr C Sundeep Babu
Assistant Professor, Department of CSE



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH.
Narayanaguda, Hyderabad, Telangana-29

2026-27



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH.

Narayanaguda, Hyderabad, Telangana - 29



2026-27

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that this is a Bonafide record of the project report titled “**A Modular Framework for Detecting and Mitigating Hallucinations in Retrieval-Augmented Generation Systems**” which is being presented as the **Project Stage-I** report in **IV year I Semester (KR23)** by

- | | |
|----------------|------------|
| 1. P.Varun Sai | 23BD1A056A |
| 2. S.Srishanth | 23BD1A0572 |
| 3. S.Aakash | 23BD1A0573 |
| 4. V.Maithresh | 23BD1A057D |

In partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Keshav Memorial Institute of Technology (An Autonomous Institution) Narayanaguda, affiliated to the Jawaharlal Nehru Technological University Hyderabad, Hyderabad

Faculty Supervisor
(Mr C Sundeep Babu)

Senior Faculty Member
(Mr P Kiran Kumar)

Head of Department
(Mr. Para Upendar)

Vision & Mission of KMIT

Vision of the Institution

- To be the fountain head in producing highly skilled, globally competent engineers.
- Producing quality graduates trained in the latest software technologies and related tools and striving to make India a world leader in software products and services.

Mission of the Institution

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi modal platforms and prepares students to become successful professionals.
- To establish an industry institute Interaction to make students ready for the industry.
- To provide exposure to students on the latest hardware and software tools.
- To promote research-based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises.
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students.

Vision & Mission of CSE

Vision of the Department

To be among the region's premier teaching and research Computer Science and Engineering departments producing globally competent and socially responsible graduates in the most conducive academic environment.

Mission of the Department

- To provide faculty with state of the art facilities for continuous professional development and research, both in foundational aspects and of relevance to emerging computing trends.
- To impart skills that transform students to develop technical solutions for societal needs and inculcate entrepreneurial talents.
- To inculcate an ability in students to pursue the advancement of knowledge in various specializations of Computer Science and Engineering and make them industry-ready.
- To engage in collaborative research with academia and industry and generate adequate resources for research activities for seamless transfer of knowledge resulting in sponsored projects and consultancy.
- To cultivate responsibility through sharing of knowledge and innovative computing solutions that benefit the society-at-large.
- To collaborate with academia, industry and community to set high standards in academic excellence and in fulfilling societal responsibilities

PROGRAM OUTCOMES (POs)

PO1. Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem Analysis: Identify formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences

PO3. Design/Development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct Investigations of Complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern Tool Usage: Create select, and, apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6. The Engineer and Society: Apply reasoning informed by contextual knowledge to societal, health, safety. Legal und cultural issues and the consequent responsibilities relevant to professional engineering practice.

PO7. Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. Life-Long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: An ability to analyze the common business functions to design and develop appropriate Information Technology solutions for social upliftments.

PSO2: Shall have expertise on the evolving technologies like Python, Machine Learning, Deep learning, IOT, Data Science, Full stack development, Social Networks, Cyber Security, Mobile Apps, CRM, ERP, Big Data, etc.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: To imbibe analytical and professional skills for successful careers and create enthusiasts to pursue advance education supplementing their career growth.

PEO2: Graduates will solve real time problems design, develop and implement innovative ideas by applying their computer engineering principles.

PEO3: Graduates will develop necessary skillset for industry by imparting state of art technology in various areas of computer science engineering.

PEO4: Graduates will engage in lifelong learning and be able to work collaboratively exhibiting high level of professionalism.

DECLARATION

We hereby declare that the results embodied in the dissertation entitled “**TrustRAG: A Modular Framework for Detecting and Mitigating Hallucinations in Retrieval-Augmented Generation Systems**” has been carried out by us together during the academic year 2026-27 as a partial fulfillment of the award of the B.Tech degree in Computer Science and Engineering from Keshav Memorial Institute of Technology (An Autonomous Institution) Narayanaguda, affiliated to JNTUH. We have not submitted this report to any other university or organization for the award of any other degree.

Student Name

Roll no.

P. VARUN SAI

23BD1A056A

S. SRISHANTH

23BD1A0572

S. AAKASH

23BD1A0573

V. MAITHRESH

23BD1A057D

ACKNOWLEDGEMENT

We take this opportunity to thank all the people who have rendered their full support to our project work. We render our thanks to **Dr. B L Malleswari**, Principal who encouraged us to do the Project.

We are grateful to **Mr. Neil Gogte**, Founder & Director, **Mr. S. Nitin**, Director for facilitating all the amenities required for carrying out this project.

We express our sincere gratitude to **Ms. Deepa Ganu**, Director Academic for providing an excellent environment in the college.

We are also thankful to **Mr. Para Upendar**, Head of the Department for providing us with time to make this project a success within the given schedule.

We are also thankful to our Faculty Supervisor **Mr C Sundeep Babu**, for her/his valuable guidance and encouragement given to us throughout the project work.

We would like to thank the entire CSE Department faculty, who helped us directly and indirectly in the completion of the project.

We sincerely thank our friends and family for their constant motivation during the project work.

Student Name	Roll no.
P. VARUN SAI	23BD1A056A
S. SRISHANTH	23BD1A0572
S. AAKASH	23BD1A0573
V. MAITHRESH	23BD1A057D

ABSTRACT

Retrieval-Augmented Generation (RAG) grounds the output of Large Language Models (LLMs) in retrieved documents, yet conventional RAG pipelines cannot tell whether a generated answer is actually supported by the evidence behind it. TrustRAG is a claim-level verified, multi-tenant RAG platform that closes this gap by treating the pipeline as a closed-loop reliability system. Users upload documents (PDF, TXT, MD, DOCX, CSV, JSON, HTML) into private Knowledge Bases, which are chunked and indexed with both dense and sparse vectors. A query is answered through hybrid retrieval fused with Reciprocal Rank Fusion, temporal-validity filtering and a SHA-256 integrity audit of the evidence, followed by grounded generation with Google Gemini. The answer is decomposed into atomic claims, each verified against the evidence by batch Natural Language Inference as SUPPORTED, CONTRADICTED or NEUTRAL, and a reliability score is computed. When reliability is low, a bounded LangGraph recovery loop rewrites the query or expands retrieval before the answer is labelled TRUSTED, UNCERTAIN, FAILED or ABSTAINED. The platform is built with a React front end, a FastAPI back end, MongoDB and Qdrant, and provides real-time execution tracing through Server-Sent Events, strict per-user data isolation and exportable audit records. This report presents the introduction, literature survey, software requirement specification and system design of TrustRAG.

LIST OF FIGURES

S.No	Name of Screenshot	Page No.
1.	Fig 1.1: System Architecture of TrustRAG	4
2.	Fig 4.1: Use Case Diagram of TrustRAG	18
3.	Fig 4.2: Sequence Diagram for Query Analysis (UC-03)	20
4.	Fig 4.3: State Chart Diagram of an Analysis Run	21
5.	Fig 4.4: Deployment Diagram of TrustRAG	22

CONTENTS

<u>DESCRIPTION</u>	<u>PAGE</u>
CHAPTER - 1	1
1. INTRODUCTION	2-4
1.1 Purpose of the project	2
1.2 Problem with Existing Systems	2
1.3 Proposed System	2
1.4 Scope of the Project	3
1.5 Architecture Diagram	3
CHAPTER - 2	5
2. LITERATURE SURVEY	6
CHAPTER - 3	7
3. SOFTWARE REQUIREMENT SPECIFICATION	8-16
3.1 Introduction to SRS	8
3.2 Role of SRS	9
3.3 Requirements Specification Document	9
3.4 Functional Requirements	9
3.5 Non-Functional Requirements	13
3.6 Performance Requirements	15
3.7 Software Requirements	15
3.8 Hardware Requirements	16
CHAPTER - 4	17
4. SYSTEM DESIGN	18-23
4.1 Introduction to UML	18
4.2 UML Diagrams	18
4.2.1 Use Case Diagram	18
4.2.2 Sequence Diagram	20
4.2.3 State Chart Diagram	21
4.2.4 Deployment Diagram	22
4.3 TECHNOLOGIES USED	23
REFERENCES	24

CHAPTER-1

1. INTRODUCTION

1.1 Purpose of the project

Conventional Retrieval-Augmented Generation (RAG) systems retrieve context and generate answers, but give no way to check whether the answer is actually supported by the retrieved evidence. When the retriever surfaces weak context, or the Large Language Model (LLM) goes beyond it, the answer still looks equally confident. This failure, known as **hallucination**, is hard to detect because standard pipelines do not audit or score their own output.

TrustRAG turns the RAG pipeline into a closed-loop reliability system. It splits each answer into atomic claims, verifies every claim against the evidence using batch Natural Language Inference (NLI), checks evidence integrity with SHA-256 hashes, and computes a reliability score. If the score is low, a bounded LangGraph recovery loop rewrites the query or expands retrieval before the answer is labelled TRUSTED, UNCERTAIN, FAILED or ABSTAINED.

1.2 Problem with Existing Systems

- **No verification of grounding:** generated answers are never compared with the evidence they claim to rest on.
- **Uniform confidence:** grounded and hallucinated answers look the same; the user gets no reliability signal.
- **Opaque answers:** answers are not broken into claims, so users cannot see what supports each statement.
- **Weak retrieval and stale content:** a single retrieval mode misses relevant text, and expired documents are used as if current.
- **No integrity checks, recovery or abstention:** tampered chunks go unnoticed, and poor retrieval leads straight to a poor answer.

1.3 Proposed System

TrustRAG is a web-based, multi-tenant platform. For every query it runs the following stages:

1. **Hybrid retrieval:** dense and sparse (BM25-style) search fused with Reciprocal Rank Fusion, plus temporal-validity filtering.
2. **Integrity audit:** each evidence chunk's SHA-256 hash is compared with the hash stored at ingestion; corrupted chunks are excluded.
3. **Grounded generation:** the Gemini LLM answers using only the selected evidence.

4. **Claim verification:** the answer is split into at most 20 claims, each classified as SUPPORTED, CONTRADICTED or NEUTRAL.
5. **Reliability scoring:** score = coverage $\times$ (1 – contradiction rate), compared with thresholds 0.80 coverage and 0.20 contradiction.
6. **Recovery and labelling:** on failure the system retries up to twice, then labels the result TRUSTED, UNCERTAIN, FAILED or ABSTAINED.

Outcome	Condition
TRUSTED	Coverage $\geq$ 0.80 and contradiction rate $\leq$ 0.20.
UNCERTAIN	Thresholds failed after recovery, score $\geq$ 0.50; answer returned with a low-confidence label.
FAILED	Thresholds failed after recovery, score $<$ 0.50; answer returned with a FAILED label.
ABSTAINED	Generator output "ABSTAIN" (empty context, insufficiency or error); no answer shown as fact.

The platform also provides per-user Knowledge Bases with isolated Qdrant collections, JWT authentication, real-time execution tracing through Server-Sent Events (SSE), and exportable audit records.

1.4 Scope of the Project

In scope:

- User registration and JWT authentication, with per-user Knowledge Bases (KBs).
- Upload and indexing of PDF, TXT, MD, DOCX, CSV, JSON and HTML documents (up to 20 MB).
- Hybrid retrieval, grounded answer generation, claim-level verification and reliability scoring with bounded recovery.
- Evidence, claim and reliability visualization with real-time tracing, and audit export.

Out of scope: sharing a KB across users, training or hosting of models, and native mobile applications.

1.5 Architecture Diagram

TrustRAG has a React frontend (Cloudflare Pages), a FastAPI backend (Render.com), MongoDB for application data, Qdrant for dense and sparse vectors, and the Google Gemini API for generation, claim decomposition and NLI. Figure 1.1 shows the architecture.

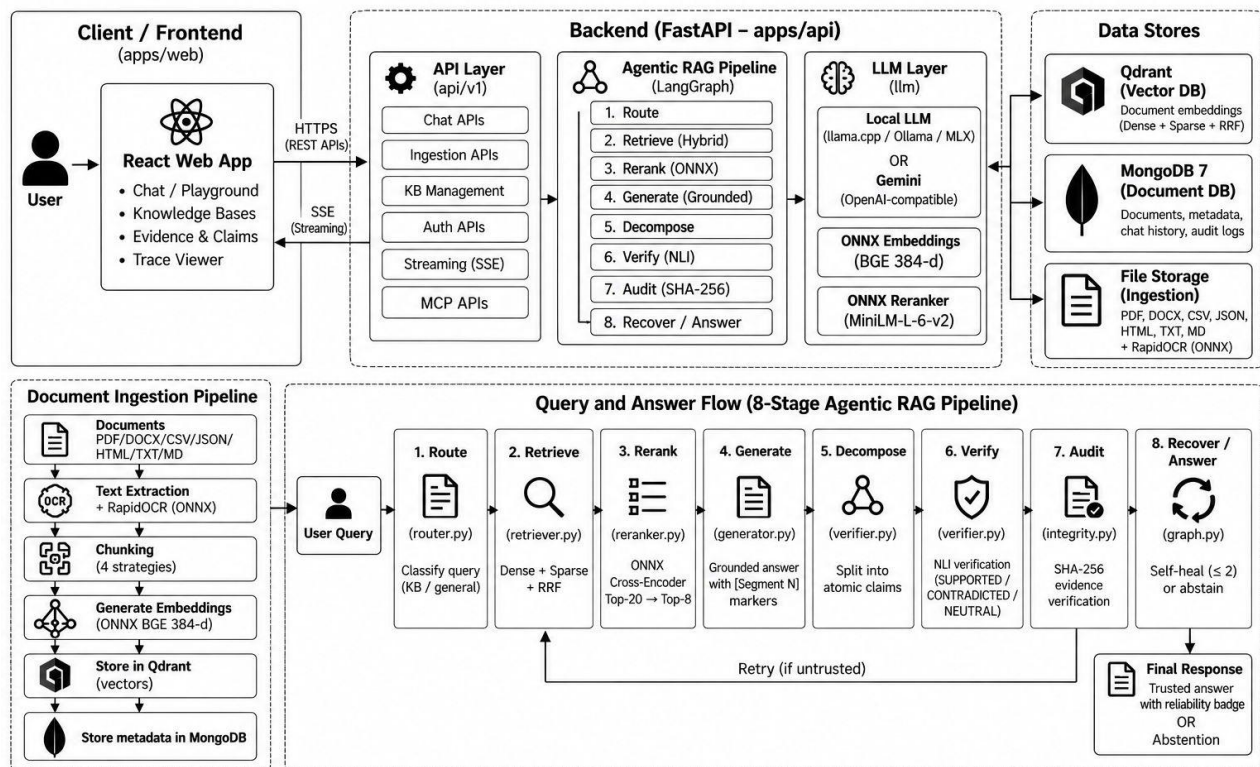


Fig 1.1: System Architecture of TrustRAG

CHAPTER-2

2. LITERATURE SURVEY

The work below covers retrieval, rank fusion, hallucination detection and agentic orchestration, and shows where TrustRAG fits.

Work	Contribution	Relation to TrustRAG
Lewis et al. (2020), RAG [3]	Combined a generator with a retrievable document index.	Basis of retrieve-then-generate; TrustRAG adds verification.
Karpukhin et al. (2020), DPR [4]	Learned dense embeddings for passage retrieval.	Basis of dense retrieval; paired with sparse search.
Robertson and Zaragoza (2009), BM25 [5]	Lexical term-weighting functions.	Basis of TrustRAG's sparse vectors.
Cormack et al. (2009), RRF [6]	Rank-based fusion of ranked lists.	Fuses dense and sparse rankings ($k = 60$).
Ji et al. (2023), Hallucination survey [7]	Causes and mitigation of hallucination.	Motivates explicit hallucination checks.
Manakul et al. (2023), SelfCheckGPT [8]	Detects hallucination via sample consistency.	Needs many samples; TrustRAG checks claims against evidence.
Min et al. (2023), FActScore [9]	Scores text by supported atomic facts.	TrustRAG applies atomic claims at run time.
Asai et al. (2024), Self-RAG [10]	Fine-tuned model that critiques itself.	TrustRAG needs no fine-tuning.
Yan et al. (2024), Corrective RAG [11]	Evaluates retrieval quality and corrects it.	TrustRAG recovers on claim-level results, bounded to 2 attempts.
Es et al. (2024), RAGAs [12]	Reference-free RAG evaluation metrics.	Offline evaluation; TrustRAG verifies every query online.
LangGraph [13]	Graph-based orchestration of LLM workflows.	Implements the retrieval-to-recovery state machine.

Research gap. Existing work tackles retrieval, fusion, hallucination metrics or self-critique separately. TrustRAG combines hybrid retrieval, SHA-256 integrity auditing, claim-level NLI verification, a bounded recovery loop, trust labelling and multi-tenant isolation in one deployable system.

CHAPTER-3

3. SOFTWARE REQUIREMENT SPECIFICATION

3.1 Introduction to SRS

A Software Requirements Specification (SRS) formally defines what a system shall do. This SRS defines TrustRAG as implemented, so that its behaviour can be verified against a fixed specification by evaluators, developers and future maintainers. It follows the IEEE 830-1998 structure and was prepared from the project source code. Key terms are given below.

Term	Definition
RAG	Retrieval-Augmented Generation: conditioning LLM output on retrieved documents.
LLM	Large Language Model (Google Gemini, in TrustRAG).
BM25 / Sparse Retrieval	Lexical retrieval via term-frequency sparse vectors with sublinear TF scaling and zone weighting.
Dense Retrieval	Vector-similarity retrieval over 384-dimensional embeddings.
RRF	Reciprocal Rank Fusion. $RRF_score(d) = 1/(rank_dense(d) + k) + 1/(rank_sparse(d) + k)$, 1-based rank, $k = 60$ (configured <code>rrf_k</code>).
NLI	Natural Language Inference: classification of a claim against evidence as SUPPORTED, CONTRADICTED or NEUTRAL.
Claim	An atomic, independently verifiable statement decomposed from a generated answer, capped at 20 per analysis (<code>max_verification_claims</code>).
Evidence	A retrieved chunk with provenance metadata and SHA-256 integrity status.
Reliability Score	$coverage \times (1 - contradiction_rate)$, clamped to $[0, 1]$; does not incorporate integrity results directly.
Reliability Verdict	Binary PASS/FAIL outcome from comparing coverage and contradiction rate with <code>minimum_evidence_coverage</code> (0.80) and <code>maximum_contradiction_rate</code> (0.20).
abstain_below	Threshold (0.50) used only to label a post-recovery substantive answer UNCERTAIN or FAILED; it does not trigger abstention itself.
Provenance	Traceable origin of evidence via SHA-256 hash comparison against ingestion-time reference hashes.
Knowledge Base (KB)	A user-owned, isolated container scoping documents and their vector/lexical indices. There is no per-KB model or threshold configuration.
Recovery	Bounded query-rewrite / expanded-retrieval loop, maximum 2 attempts, triggered on a FAIL verdict.
Abstention (ABSTAIN)	The literal answer string produced by the generator on empty context, LLM self-judged insufficiency, or a generation exception; the only path to <code>reliability.status = ABSTAINED</code> .
LangGraph / StateGraph	Graph-based orchestration of the retrieval $\rightarrow$ generation $\rightarrow$ verification $\rightarrow$ recovery pipeline (<code>app/agent/graph.py</code>).
JWT	HS256-signed bearer token, required on all endpoints except <code>/health</code> , <code>/auth/register</code> and <code>/auth/login</code> .
Analysis (Analysis Run)	One query execution through the full pipeline, persisted in the <code>analyses</code> collection.
Multi-Tenancy	Per-user data isolation across MongoDB (<code>user_id</code> scoping) and Qdrant (per-KB collections).

3.2 Role of SRS

- Gives developers, evaluators and maintainers one unambiguous description of what TrustRAG shall do.
- Is the basis for the design in Chapter 4 and for verification, since each requirement has stated behaviour and failure conditions.
- Controls scope: unimplemented features (for example per-KB configuration) are stated as not implemented.

3.3 Requirements Specification Document

Users. The only in-app role is the **Authenticated User**, who can access only the resources they created. A System Operator configures deployment (environment variables, config/models.yaml) but has no special in-app role.

Constraints and assumptions.

- Backend in Python 3.11 with FastAPI and Pydantic v2; frontend in React 18 with Vite 6; orchestration with LangGraph.
- MongoDB stores application data and Qdrant stores vectors; the Google Gemini API is a hard dependency with no alternative provider.
- Authentication is JWT (HS256) with Bcrypt (12 rounds); recovery is capped at 2 attempts.
- Single-instance deployment and an evergreen browser (ES2020+, EventSource) are assumed, not verified.

3.4 Functional Requirements

The 56 functional requirements (FR-01 to FR-56) are summarized below by group.

IDs	Requirement	Behaviour
FR-01 – 04	Registration, login and profile	Register with email, password and name (Bcrypt-hashed, 20/min limit); log in to receive a JWT (HS256, configurable expiry); view own profile. Errors: 409, 422, 401, 429.
FR-05 – 06	Access control and rate limiting	Every resource request is checked server-side against the owner's user_id (anti-IDOR, 403/404). Registration is limited to 20/min and analysis creation to 10/min.
FR-07 – 10	Knowledge Base management	Create and list KBs; delete a KB with its documents, chunks and vectors (sequential, not atomic); each KB has its own Qdrant collection kb_{kb_id}.
FR-11 – 12	Upload and parsing	Accept PDF, TXT, MD, DOCX, CSV, JSON, HTML and HTM up to 20 MB, checked against config/models.yaml; extract text by format; parse failure marks the document failed.

IDs	Requirement	Behaviour
FR-13 – 18	Text processing	Normalize text (NFKD, de-hyphenation), weight Title/Header/Body zones (2.0/1.5/1.0), Porter-stem, chunk (512 characters, 64 overlap), extract Effective-from/until dates, and store a SHA-256 hash per chunk.
FR-19 – 21	Indexing	Create 384-dimensional embeddings (Gemini by default) and BM25-style sparse vectors; store both in the KB's Qdrant collection with user_id and hash in the payload.
FR-22 – 23	Document management	List documents of a KB; delete a document with its chunks and vectors.
FR-24	Query submission	Start an asynchronous analysis run (background task); an empty KB completes with an informational message.
FR-25 – 30	Retrieval	Dense and sparse search, RRF fusion ($k = 60$), temporal-validity filtering, optional cross-encoder reranking (off by default), and top-k evidence selection.
FR-31 – 34	Generation and verification	Generate a grounded answer from evidence only (retrieved text treated as untrusted; ABSTAIN on empty context or error); decompose into claims; batch NLI (fallback per claim); link claims to evidence.
FR-35 – 39	Integrity and reliability	Recompute SHA-256 and exclude CORRUPTED chunks; compute coverage and contradiction rate; PASS/FAIL verdict from thresholds 0.80 and 0.20; diagnose EVIDENCE_CONFLICT, LOW_COVERAGE or RETRIEVAL_FAILURE.
FR-40 – 43	Recovery	Attempt 1 rewrites the query; attempt 2 doubles top_k to 40; at most 2 attempts (3 pipeline cycles); generation and verification are re-run after each.
FR-44	Terminal outcome	ABSTAINED if the answer is exactly "ABSTAIN"; otherwise completed and labelled TRUSTED, UNCERTAIN (score ≥ 0.50) or FAILED (score < 0.50).
FR-45 – 47	Analysis records	Persist each run (query, status, verdict, answer, configuration); list a user's analyses; return one analysis to its owner.
FR-48	Execution trace	Stream stage events over SSE, authenticated by a token query parameter; ping every 3 s; end on a terminal event or after 120 s of silence.
FR-49 – 53	Results and export	Return claims, evidence with integrity status, and conflicts across the user's analyses; export an audit record as JSON or JSON-LD.
FR-54 – 56	Evaluation and health	Record, list and fetch evaluation experiments; a public health endpoint reports MongoDB, Qdrant, models and supported formats (always HTTP 200).

Known gap (FR-44). Recovery exhaustion does not force abstention: a substantive but low-reliability answer is still returned, labelled FAILED or UNCERTAIN. The 0.50 threshold only chooses between these two labels, and an empty KB is currently labelled TRUSTED with score 0.0.

3.4.1 Core Pipeline Requirements in Detail

ID	Requirement	Description and Behaviour	Inputs / Outputs	Conditions and Exceptions
FR-24	Query Submission for Analysis	Accept a natural-language query against a KB and start an asynchronous analysis run: create an analyses record with initial status and trigger the LangGraph state machine as a background task. Rate-limited to 10 requests/minute. If the KB's collection holds zero indexed points the request is not rejected: the analysis completes with an informational message, status = completed, reliability.status = TRUSTED, reliability.score = 0.0 (see Section 3.9).	In: KB identifier, query text. Out: analysis resource with id and initial status.	Pre: KB exists and is owned by the user. Exc: 429; 403/404.
FR-27	Hybrid Fusion via RRF	Combine the dense and sparse rankings into one fused ranking: $RRF_score(d) = 1/(rank_dense(d) + k) + 1/(rank_sparse(d) + k)$, 1-based rank, $k = 60$ (rrf_k), sorted descending (reciprocal_rank_fusion() in retriever.py).	In: dense ranking, sparse ranking. Out: single fused ranking.	—
FR-31	Grounded Answer Generation	Generate an answer with the configured Gemini LLM, constrained to the selected evidence, with retrieved content explicitly treated as untrusted data rather than instructions (GROUNDING_SYSTEM_PROMPT in generator.py).	In: query text, evidence set. Out: answer text, or the literal string "ABSTAIN".	Exc: any exception during LLM invocation is caught and the answer is set to "ABSTAIN" rather than propagating an error or fabricating content.
FR-33	Batch NLI Claim Verification	Verify decomposed claims against the evidence, up to 20 per analysis (cost_controls.max_verification_claims), using a single batch LLM call by default and classifying each as SUPPORTED, CONTRADICTED or NEUTRAL. Falls back to per-claim calls if the batch call raises an exception.	In: claims (capped at 20), evidence set. Out: per-claim verdict and cited evidence ids, stored in claims.	Exc: claims beyond the cap are excluded before verification runs.
FR-35	SHA-256 Integrity Audit	For each evidence chunk, recompute its SHA-256 hash and compare it with the reference hash stored at ingestion (FR-18) (integrity.py). This establishes integrity relative to the ingestion-time reference, not factual accuracy.	In: chunk content, stored reference hash. Out: per-chunk status VERIFIED / CORRUPTED, stored with the evidence record.	Exc: a mismatch marks the evidence CORRUPTED and excludes it from the evidence set used for generation.
FR-38	Reliability Verdict Determination	Compare coverage with minimum_evidence_coverage (0.80) and contradiction rate with maximum_contradiction_rate (0.20), both read from config/models.yaml. The reliability score is $coverage \times (1 - contradiction_rate)$, clamped to $[0, 1]$. Integrity results do not enter the formula; they act upstream by excluding non-VERIFIED chunks from the evidence set.	In: coverage, contradiction rate, integrity results. Out: verdict PASS (proceed to answer) or FAIL (proceed to diagnosis/recovery).	Exc: the configuration also defines abstain_below (0.50) and three weights (citation_correctness_weight, evidence_coverage_weight, source_integrity_weight); the weights are unused, reserved configuration (Section 3.9).
FR-42	Recovery Attempt Ceiling Enforcement	Allow no more than max_recovery_attempts (2) per analysis, so the recovery loop is finite. A run has at most three pipeline cycles: one initial cycle plus up to two recovery repeats. At the ceiling the state machine terminates with the answer, verdict and diagnosis that currently exist (should_recover()).	In: current attempt count. Out: termination state (see FR-44).	—

ID	Requirement	Description and Behaviour	Inputs / Outputs	Conditions and Exceptions
FR-44	Terminal Outcome Determination	After the LangGraph run terminates: (a) if the answer is exactly "ABSTAIN" (empty evidence, LLM self-judged insufficiency, or generation exception), record status = abstained and reliability.status = ABSTAINED, surfacing no answer text as fact; (b) otherwise record status = completed and surface the answer regardless of thresholds, labelled TRUSTED (thresholds met), UNCERTAIN (score $\geq$ abstain_below = 0.50 despite failing) or FAILED (score < 0.50).	In: exhausted or passing pipeline state. Out: final status and reliability.status, plus answer text except in case (a).	Post: the analysis record reflects the outcome. Exc: IMPLEMENTATION GAP: recovery exhaustion does not force abstention; a substantive low-reliability answer is returned labelled FAILED/UNCERTAIN (Section 3.9).
FR-48	Real-Time Execution Trace Streaming	Stream live stage telemetry for an in-progress run through SSE, authenticated by a token query parameter (EventSource cannot set custom headers). A ping heartbeat is sent after every 3 seconds of inactivity; the stream ends after 120 consecutive seconds without a new event or on a terminal event (analysis.completed, analysis.abstained, analysis.failed) (sse_event_generator).	In: analysis identifier, JWT as token query parameter. Out: SSE stream of pipeline stage transitions.	Pre: user owns the analysis.

3.4.2 Known Limitations

- **Recovery exhaustion does not force abstention (FR-44):** a substantive low-reliability answer is returned, labelled FAILED or UNCERTAIN.
- **abstain_below (0.50) does not trigger abstention;** genuine abstention occurs only when the generator outputs "ABSTAIN".
- **Empty-KB analyses are labelled TRUSTED with score 0.0,** an artifact of the current verification logic that should be treated as a defect.
- **Unused configuration:** the three reliability weights in models.yaml are not used; the score is coverage $\times$ (1 – contradiction rate).
- **Integrity is not part of the score:** SHA-256 results only filter evidence and feed the Conflicts view.
- **Deletion is not atomic** across MongoDB and Qdrant, and a failure part-way may leave partial data.
- **Security gaps:** no Content-Security-Policy or HSTS header, a broad CORS regex rule, and no protection against malicious content inside accepted file types.

3.4.3 Requirements Traceability

Requirements are mapped to modules under apps/api; 'Test' is cited only where a test file exists among the 13 files in apps/api/tests.

Requirements	Component / Module	Verification
FR-01 – 06	app/api/v1/auth.py, app/services/auth_service.py, app/core/security.py, app/core/rate_limiter.py	Test (test_auth.py) + Inspection
FR-07 – 10, 22 – 23	app/api/v1/knowledge_bases.py, app/api/v1/documents.py, app/services/kb_service.py, app/db/qdrant.py	Test (test_kb.py) + Inspection

Requirements	Component / Module	Verification
FR-11 – 21	app/ingestion/ (parser, preprocessor, chunker, sparse_vector, pipeline), app/core/model_registry.py	Test (test_ingestion.py, test_preprocessor.py)
FR-24 – 30	app/api/v1/analyses.py, app/retrieval/retriever.py, app/retrieval/reranker.py	Test (test_analyses.py, test_retrieval.py)
FR-31 – 34	app/generation/generator.py, app/verification/verifier.py	Test (test_generation.py, test_verification.py)
FR-35 – 44	app/verification/integrity.py, app/agent/graph.py, app/services/analysis_service.py	Test (test_integrity.py, test_agent.py)
FR-45 – 53	app/api/v1/analyses.py, claims.py, evidence.py, conflicts.py, app/services/analysis_service.py	Test (test_analyses.py) + Inspection
FR-54 – 56	app/api/v1/experiments.py, app/api/v1/health.py, app/services/experiment_service.py	Test (test_experiments.py, test_health.py)

3.5 Non-Functional Requirements

3.5.1 Security Requirements

ID	Requirement	Specification
NFR-S1	Password Storage	Passwords shall be stored exclusively as Bcrypt hashes (cost factor 12); plaintext passwords shall never be persisted or logged.
NFR-S2	Token-Based Authentication	A valid, non-expired JWT is required for every API endpoint except GET /api/v1/health, POST /api/v1/auth/register and POST /api/v1/auth/login. All other endpoints, including GET /api/v1/auth/me, enforce authentication through Depends(get_current_user). The SSE stream endpoint differs only in mechanism: it validates a token query parameter manually because EventSource cannot set headers; it is not unauthenticated.
NFR-S3	Minimum Secret Strength	The system shall refuse to start if the configured JWT signing secret is shorter than 32 characters (Pydantic field validator on application settings).
NFR-S4	Cross-Tenant Isolation	The system shall enforce server-side ownership verification on every resource-scoped operation (KB, document, analysis, claim, evidence, experiment), independent of any client-side assumption, per FR-05.
NFR-S5	Physical Vector Data Segregation	Each KB's vector and sparse index data shall be stored in a dedicated, uniquely named Qdrant collection, preventing cross-tenant vector retrieval even if an application-layer authorization defect occurred.
NFR-S6	Untrusted Content Handling in Generation	The generation prompt shall instruct the LLM to treat all retrieved document content as untrusted data and not to follow instructions embedded in it. This is a prompt-level mitigation that reduces, but does not guarantee prevention of, prompt injection, since prompt instructions are advisory to the LLM.
NFR-S7	Context Boundary Injection Mitigation	Untrusted metadata fields (for example filenames) inserted into LLM prompts shall be sanitized by stripping non-printable characters and truncating length, reducing but not eliminating the risk of a crafted filename disrupting a context-segment delimiter.
NFR-S8	Rate Limiting	The registration endpoint shall be limited to 20 requests/minute and the analysis-creation endpoint to 10 requests/minute, enforced within the single-instance scope (no distributed rate-limit store is implemented).
NFR-S9	Production Credential Enforcement	The system shall refuse to start in production mode if no Qdrant API key is configured (Pydantic model validator on application settings).
NFR-S10	Error Detail Redaction	The system shall not expose raw internal exception messages or stack traces to API clients on pipeline failure; full detail is logged server-side and only a generic, typed error message is returned.
NFR-S11	Frontend HTTP Headers	The deployed frontend shall serve, via the Cloudflare Pages_headers file applied to all routes: X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin, and a Permissions-Policy restricting geolocation, camera and microphone. No Content-Security-Policy or Strict-Transport-Security header is present in the current configuration. This applies only to the Cloudflare Pages deployment, not local development.

ID	Requirement	Specification
NFR-S12	File Upload Format and Size Enforcement	The system shall reject uploads whose extension is not in the configured format registry, and uploads exceeding the maximum size, before any parsing (per FR-11). [NEEDS CONFIRMATION]: no protection against malicious content inside an accepted file type (for example a parser-exploiting file or CSV formula injection) is established in source; this is noted as a gap.

3.5.2 Usability Requirements

ID	Requirement	Specification
NFR-U1	Reliability Status Visibility	The frontend shall visibly present the reliability status (TRUSTED, UNCERTAIN, FAILED or ABSTAINED) of every completed or abstained analysis, not only the answer text (ReliabilityBadge component).
NFR-U2	Claim-Level Inspectability	The frontend shall allow the user to inspect the individual decomposed claims and their NLI verdicts behind any generated answer (ClaimInspector component, /analyses/{id}/claims).
NFR-U3	Evidence Traceability	The frontend shall allow the user to view the evidence chunks retrieved for an analysis, including source document, integrity status and relevance scores (EvidenceViewer component, /analyses/{id}/evidence).
NFR-U4	Live Execution Feedback	The frontend shall display real-time pipeline stage progress for an in-progress analysis through the SSE-consuming execution trace view, not a static loading indicator.
NFR-U5	Consistent Routing on Refresh	The deployed frontend shall resolve all client-side routes correctly on direct navigation or refresh, through the Cloudflare Pages SPA redirect rule (_redirects).

3.5.3 Reliability Requirements

ID	Requirement	Specification
NFR-R1	Bounded Recovery Loop	The number of recovery attempts per analysis shall be bounded by <code>max_recovery_attempts = 2</code> , guaranteeing that the loop is finite. The total number of pipeline cycles is at most three.
NFR-R2	Graceful Degradation on External Service Failure	The system shall catch exceptions from the external LLM provider and resolve to a defined fallback ("ABSTAIN" for generation failures; per-claim fallback for verification batch failures) rather than propagating an unhandled error.
NFR-R3	Data Consistency on Deletion	On deletion of a KB or document, the system executes a sequence of delete operations against MongoDB and Qdrant intended to remove all dependent data. The sequence is not transactionally atomic across the two stores; a failure between steps could leave partial or orphaned data. Only the intended, normal-path behaviour is specified.
NFR-R4	Health Observability	The system shall expose a health endpoint reporting the live status of both backing data stores, suitable for container orchestration and monitoring tooling (FR-56).
NFR-R5	Tamper Detection	The system shall detect content drift or tampering in retrieved chunks by recomputing and comparing SHA-256 hashes against reference values stored at ingestion, excluding any chunk that fails from the evidence set (FR-18, FR-35). This establishes integrity relative to the stored reference, not the factual truth of the content.
NFR-R6	Temporal Currency	The system shall exclude, at retrieval time, chunks of documents whose declared <code>effective_from</code> / <code>effective_until</code> metadata places them outside the current validity window. The filtering relies entirely on document-declared metadata extracted at ingestion; the system does not independently verify whether content is factually current.
NFR-R7	Claim-Level Grounding Verification	The system shall verify every decomposed claim of a generated answer against retrieved evidence by NLI classification, up to 20 claims per analysis (<code>cost_controls.max_verification_claims</code>). Claims beyond the limit are excluded before verification.
NFR-R8	Threshold-Gated Trust Labelling	The system shall compute a reliability score from coverage and contradiction rate and classify the answer using three independently configured thresholds: <code>minimum_evidence_coverage = 0.80</code> and <code>maximum_contradiction_rate = 0.20</code> jointly determine the PASS/FAIL verdict that drives recovery; <code>abstain_below = 0.50</code> is used only after a FAIL verdict and recovery exhaustion to choose between UNCERTAIN (score ≥ 0.50) and FAILED (score < 0.50). <code>abstain_below</code> does not cause abstention.

ID	Requirement	Specification
NFR-R9	Adaptive Recovery Before Acceptance	The system shall attempt query rewriting and/or expanded retrieval when an answer fails reliability thresholds, before finalizing its trust status (FR-40, FR-41). This does not guarantee that every user-facing answer meets the threshold or is withheld (see FR-44 and Section 3.9).

3.6 Performance Requirements

No numeric targets are asserted where the repository sets none ([NEEDS CONFIRMATION]).

ID	Requirement	Specification
NFR-P1	Non-Blocking Analysis Execution	The pipeline shall execute as a background task, decoupled from the HTTP request that starts it (POST /analyses returns immediately with a pending status; the pipeline runs via BackgroundTasks). The client is not blocked waiting for completion.
NFR-P2	Real-Time Progress Visibility	The system shall make incremental pipeline progress observable during an in-progress analysis through Server-Sent Events, rather than requiring the client to poll with no intermediate feedback.
NFR-P3	Bounded Recovery Cost	The number of additional retrieval, generation and verification cycles triggered by low reliability shall be bounded by a fixed ceiling (max_recovery_attempts = 2), preventing unbounded latency or cost for any single analysis.
NFR-P4	Verification Call Efficiency	The system shall attempt to verify all claims of an answer in a single batched LLM invocation. If the batch call raises an exception, it falls back to individual per-claim calls rather than failing the analysis. The single-batch path is the default and expected case.
NFR-P5	Ingestion Size Guarding	The system shall enforce the configured maximum upload size (20 MB) by streamed, chunked reading, rejecting oversized files before their full content is buffered in memory. [NEEDS CONFIRMATION]: no committed target exists for ingestion throughput or time-to-index for a document; none is asserted.
NFR-P6	Concurrent Request Handling	The backend is built on FastAPI's asynchronous request model (async/await, Motor async driver), which supports concurrent request handling within a single process. [NEEDS CONFIRMATION]: no concurrent-user or concurrent-analysis capacity figure is established, and under the single-instance assumption no multi-replica concurrency guarantee is claimed.

3.7 Software Requirements

Component	Production	Local / development
Frontend	React 18, Vite 6 on Cloudflare Pages	Vite dev server (localhost:5173)
Backend API	Python 3.11, FastAPI, Docker on Render.com	Uvicorn or Docker Compose (localhost:8000)
MongoDB	MongoDB Atlas (7.0+, Motor driver)	Local MongoDB / Docker
Qdrant	Qdrant Cloud (1.10.1)	Local Docker (6333/6335)
External AI	Google Gemini API (GEMINI_API_KEY)	Same

Interfaces. The frontend talks to the backend over REST/HTTPS with a Bearer JWT, and over a long-lived SSE stream (GET /api/v1/analyses/{id}/stream) with a token query parameter. The frontend has 11 pages: Login, Register, Dashboard, Playground, Knowledge Bases, Evidence, Claims, Conflicts, Experiments, Settings and Trace. The backend reaches MongoDB (Motor), Qdrant (QdrantClient) and Gemini (langchain-google-genai).

3.8 Hardware Requirements

The repository specifies no minimum CPU, RAM, storage or GPU, so none is imposed. Production uses managed services (Render.com, MongoDB Atlas, Qdrant Cloud, Cloudflare Pages) and the default Gemini cloud embeddings need no local model. Clients need only a current web browser.

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher
RAM	8 GB minimum (16 GB recommended)
GPU	NVIDIA GPU with 2+ GB VRAM
Storage	20–50 GB free disk space (SSD preferred)

CHAPTER-4

4. SYSTEM DESIGN

4.1 Introduction to UML

The Unified Modeling Language (UML) is a standard visual language for specifying and documenting software. Structural diagrams, such as the deployment diagram, show the static arrangement of a system, while behavioural diagrams, such as use case, sequence and state chart diagrams, show interaction and change over time. These four diagrams are used for TrustRAG.

Why four diagrams. Each diagram answers a different question about TrustRAG. The use case diagram shows what the system offers and to whom. The sequence diagram shows the order in which components exchange messages. The state chart diagram shows how one analysis run changes status over time. The deployment diagram shows where each part runs in production. Read together, they connect the requirements of Chapter 3 to the implemented system.

4.2 UML Diagrams

4.2.1 Use Case Diagram

The Authenticated User is the primary actor; the Google Gemini API takes part as an external system. Recovery and abstention are alternate flows inside UC-03.

Reading the diagram. The rectangle labelled TrustRAG System is the system boundary, and the seven ovals inside it are use cases UC-01 to UC-07. The stick figure outside the boundary is the Authenticated User, who is linked to every use case. The Google Gemini API is drawn outside the boundary as a <<system>> actor, because the platform calls it for generation, NLI and embeddings but does not control it. It is joined to UC-03 by a solid line and to UC-02 by a dashed line, showing that answering a query depends on Gemini directly, while document management uses it for embeddings during indexing.

Rules recorded in the note. Every use case except UC-01 requires a valid JWT, since registration and login are the only ways to obtain one. Recovery, query rewriting and abstention are shown as alternate flows inside UC-03 rather than as separate use cases, because the user never triggers them: the pipeline decides on its own after verification. This is why the UC-03 flow below lists them as alternates A, B and C.

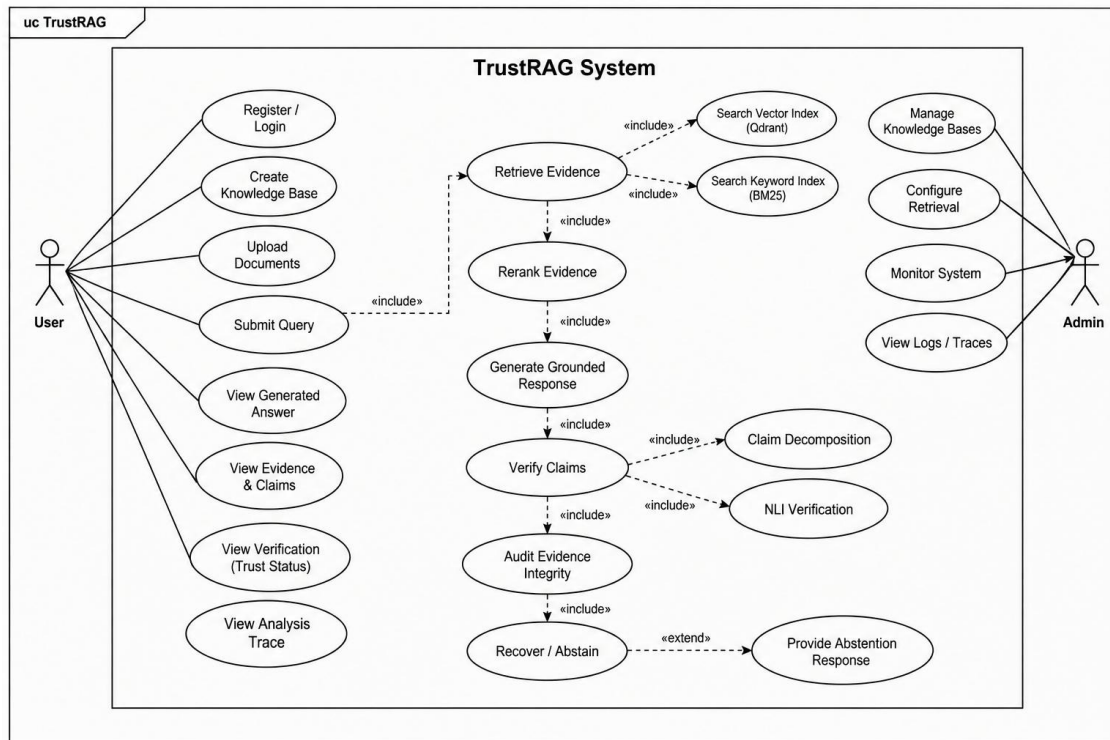


Fig 4.1: Use Case Diagram of TrustRAG

UC	Name	Purpose
UC-01	Register and Authenticate	Create an account and obtain a JWT.
UC-02	Manage KB and Documents	Create a KB and upload documents for indexing.
UC-03	Submit Query for Analysis	Ask a question and receive a reliability-assessed answer.
UC-04	Inspect Analysis Results	Review claims, evidence, integrity status and verdict.
UC-05	Review Conflicts	See contradicted claims and tampered evidence across analyses.
UC-06	Export Audit Record	Download a JSON/JSON-LD audit record of one analysis.
UC-07	Record and Review Experiments	Log and view evaluation runs.

UC-03 main flow

1. The user submits a query against a Knowledge Base.
2. The system performs hybrid retrieval and temporal filtering.
3. The system audits evidence integrity and excludes corrupted chunks.
4. The system generates a grounded answer from verified evidence.
5. The system decomposes the answer into claims and verifies them by batch NLI.
6. The system computes the reliability verdict; on PASS the result is TRUSTED.

Alternate flows

- **A, recovery:** on FAIL with attempts left, the system rewrites the query or expands retrieval and repeats steps 2 to 6.

- **B, recovery exhausted:** a non-ABSTAIN answer is returned labelled UNCERTAIN or FAILED.
- **C, abstention:** empty evidence, LLM-judged insufficiency or a generation error gives ABSTAINED with no answer shown as fact.

UC-03 summary. The user submits a query; the system retrieves and filters evidence, audits integrity, generates a grounded answer, verifies its claims and computes the reliability verdict. On PASS the result is TRUSTED. On FAIL the system recovers (up to 2 attempts); if still failing, the answer is returned as UNCERTAIN or FAILED, and if the generator abstains the result is ABSTAINED.

4.2.2 Sequence Diagram

Figure 4.2 shows the message flow of UC-03. The browser receives the pending analysis immediately and follows progress over SSE while the pipeline runs in the background.

Reading the diagram. Each box at the top is a participant (User/Browser, FastAPI Service, LangGraph Pipeline, Qdrant, Gemini LLM and MongoDB) with a dashed lifeline below it. Solid arrows are requests, dashed arrows are replies, and the numbers give the order of messages. The dashed frame marked loop covers the steps that repeat while the verdict is FAIL and fewer than 2 recovery attempts have been used.

Walkthrough. Steps 1 to 5 are the asynchronous hand-off. The browser posts the query, FastAPI inserts an analysis record with status pending and returns it at once (steps 2 and 3), the browser opens the SSE stream (step 4), and the LangGraph run starts as a background task (step 5). Inside the loop the pipeline searches Qdrant, fuses and filters the candidates, reads the reference hashes from MongoDB to audit the evidence, asks Gemini for a grounded answer, decomposes it into claims, verifies them by batch NLI, and computes coverage, contradiction rate and the PASS/FAIL result (steps 6 to 11). If the answer is "ABSTAIN", the generator returns that string instead of an answer. On FAIL, the recovery step rewrites the query (attempt 1) or doubles top_k to 40 (attempt 2) and the loop repeats. Steps 12 and 13 persist the outcome in MongoDB and send the terminal SSE event, so the user's request is never blocked while the pipeline runs.

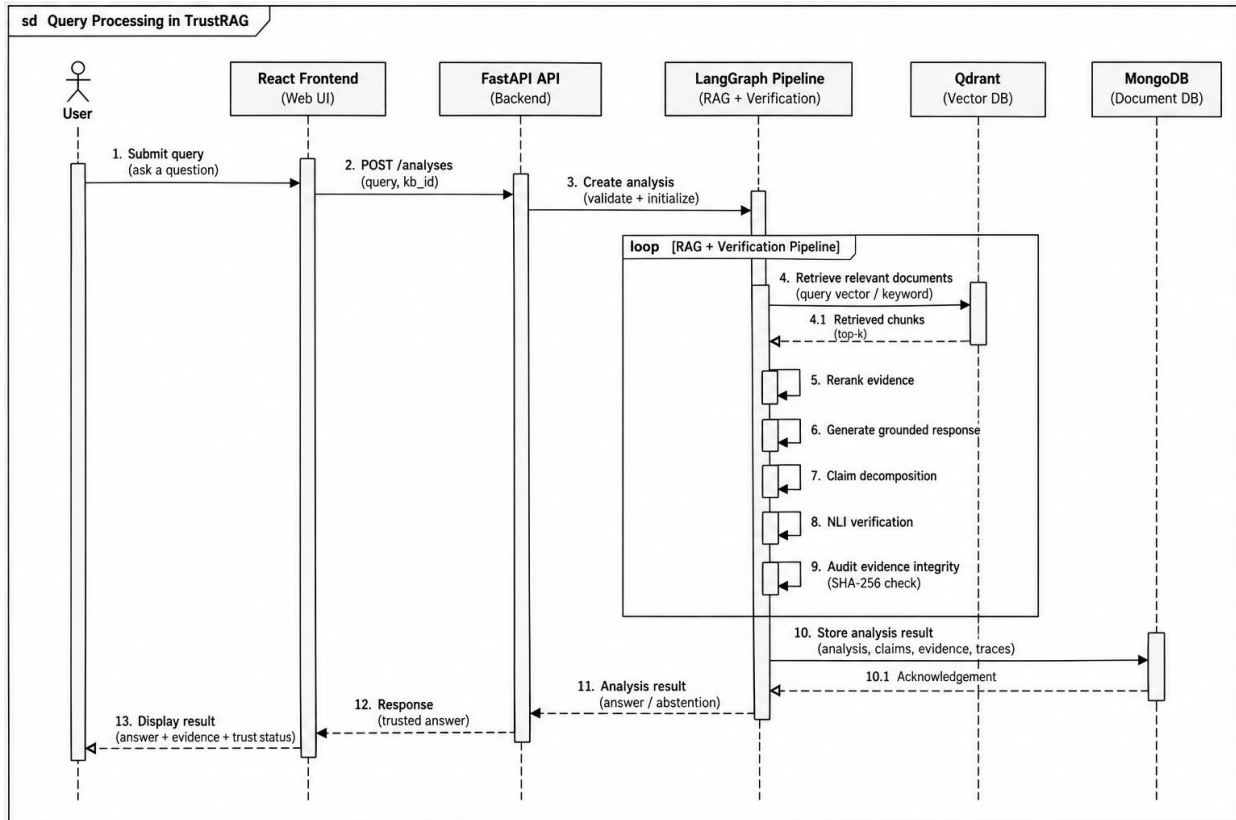


Fig 4.2: Sequence Diagram for Query Analysis (UC-03)

4.2.3 State Chart Diagram

Figure 4.3 shows the lifecycle of an analysis run: Pending, Running (with the recovery loop), and the terminal states Completed (TRUSTED or UNCERTAIN/FAILED), Abstained and Failed.

Reading the diagram. The filled circle is the initial state and the ringed circle is the final state. Rounded boxes are states and labelled arrows are transitions. A run starts in Pending when the analysis record is created and enters Running when the pipeline starts. Running is a composite state made of six sub-states: Retrieving, Auditing Integrity, Generating Answer, Verifying Claims, Scoring Reliability and Recovering.

Exits from Running. On PASS, the run ends as Completed with reliability TRUSTED. If scoring fails and no attempts remain, it ends as Completed with UNCERTAIN (score of 0.50 or more) or FAILED (score below 0.50). If the generator returns "ABSTAIN", the run moves straight to Abstained. A FAIL with fewer than 2 attempts used sends the run through Recovering and back to Retrieving; this is the only cycle in the chart, and its limit keeps the loop finite. Any unhandled error moves the run to Failed, shown by the dashed red arrow. The reliability label FAILED belongs to a completed run, whereas the Failed state means the pipeline itself raised an error. Every path ends at the final state.

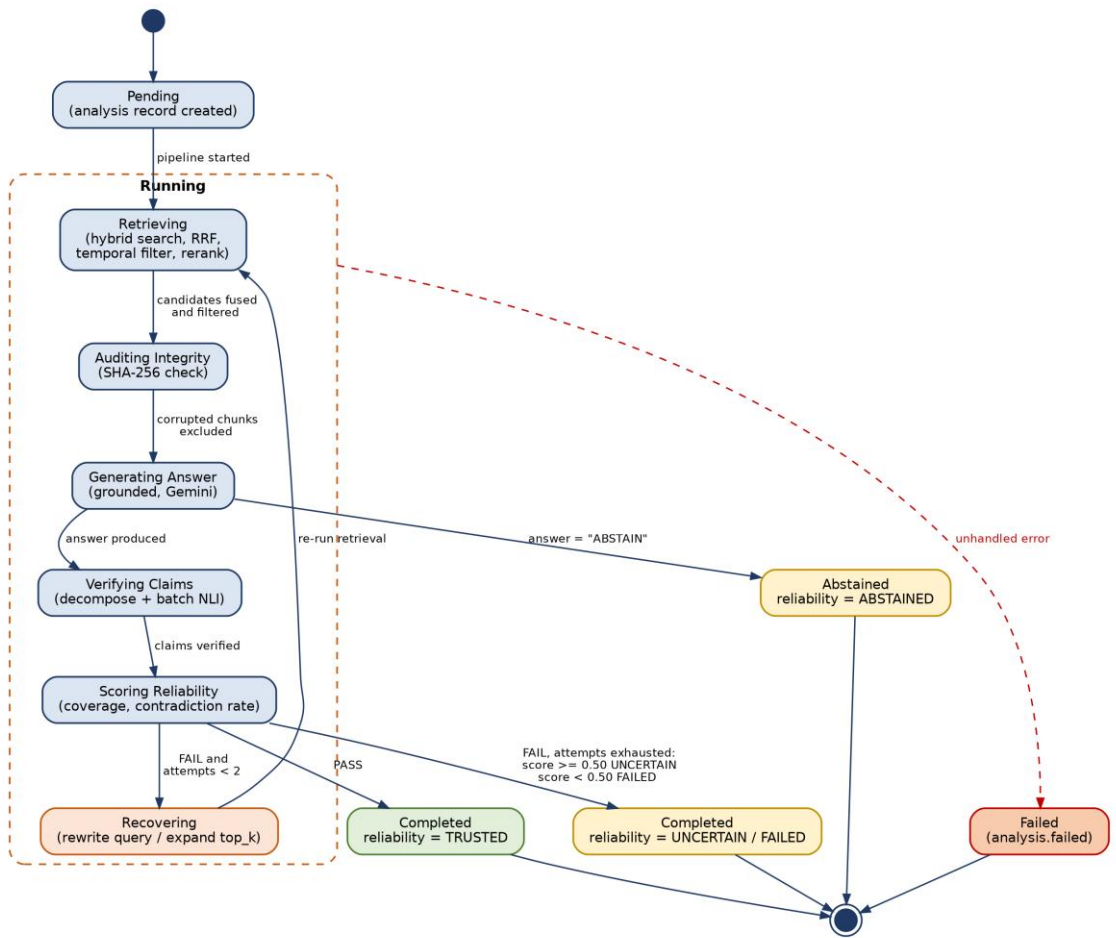


Fig 4.3: State Chart Diagram of an Analysis Run

4.2.4 Deployment Diagram

Figure 4.4 maps the software onto the production hosting: the browser loads the SPA from Cloudflare Pages and calls the Docker backend on Render.com, which uses MongoDB Atlas, Qdrant Cloud and the Gemini API.

Reading the diagram. Each 3-D box is a node with its stereotype: <<device>> for the client, <<execution environment>> for Cloudflare Pages, <<container>> for Render.com, <<database>> for MongoDB Atlas, <<vector store>> for Qdrant Cloud and <<external service>> for the Gemini API. The lines are communication links labelled with their protocol.

Walkthrough. The browser runs the React 18 single-page application, loads its static assets over HTTPS from Cloudflare Pages (which also applies the `_redirects` and `_headers` files), and then calls the backend directly over HTTPS for REST requests and the SSE stream. The backend runs as a Docker image (python:3.11-slim) on Render.com and hosts FastAPI with Uvicorn, the LangGraph pipeline, SlowAPI rate limiting, JWT and Bcrypt. It reaches MongoDB Atlas through the Motor driver, Qdrant Cloud over HTTP/gRPC, and the Gemini API over HTTPS. The note in the figure records the local setup: the Vite dev server on localhost:5173, Uvicorn or Docker Compose on

localhost:8000, and local MongoDB and Qdrant containers. Because the frontend, backend and data services are deployed separately, each can be changed without redeploying the others.

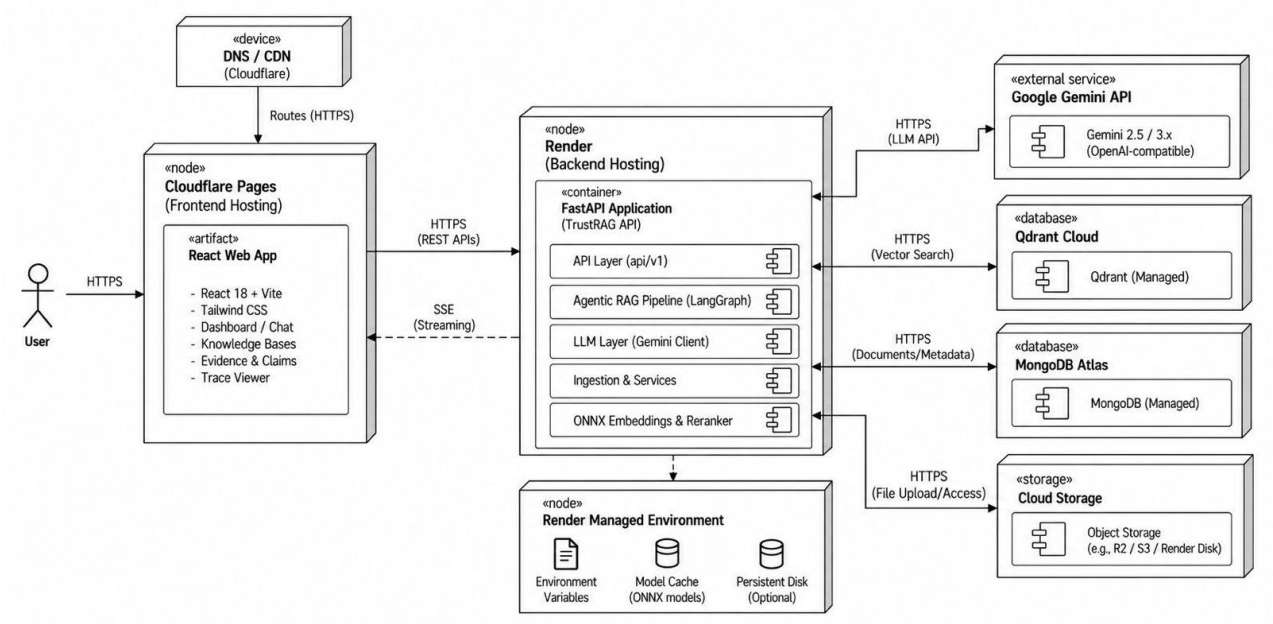


Fig 4.4: Deployment Diagram of TrustRAG

4.3 TECHNOLOGIES USED

Layer	Technology
Frontend	React 18, Vite 6, Tailwind CSS; hosted on Cloudflare Pages
Backend	Python 3.11, FastAPI, Pydantic v2, Uvicorn; Docker on Render.com
Orchestration and LLM	LangGraph, LangChain (langchain-google-genai), Google Gemini API
Embeddings and reranking	Gemini embedding-001 (default) or all-MiniLM-L6-v2; optional CrossEncoder
Retrieval	Dense search, BM25-style sparse vectors, RRF (k = 60), Porter stemmer
Data stores	MongoDB (Motor), Qdrant
Security and integrity	JWT (HS256), Bcrypt, SlowAPI rate limiting, SHA-256
Streaming and testing	Server-Sent Events; backend test suite of 13 test files

REFERENCES

- [1] TrustRAG source repository, <https://github.com/MaithreshVaddi-27/TrustRAG> (primary source for all implementation claims in this report).
- [2] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- [3] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems* 33 (NeurIPS 2020).
- [4] Karpukhin, V., Oguz, B., Min, S., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *Proceedings of EMNLP 2020*.
- [5] Robertson, S., and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4).
- [6] Cormack, G. V., Clarke, C. L. A., and Buettcher, S. (2009). Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods. *Proceedings of SIGIR '09*.
- [7] Ji, Z., Lee, N., Frieske, R., et al. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12).
- [8] Manakul, P., Liusie, A., and Gales, M. J. F. (2023). SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. *Proceedings of EMNLP 2023*.
- [9] Min, S., Krishna, K., Lyu, X., et al. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *Proceedings of EMNLP 2023*.
- [10] Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *ICLR 2024*.
- [11] Yan, S.-Q., Gu, J.-C., Zhu, Y., and Ling, Z.-H. (2024). Corrective Retrieval Augmented Generation. *arXiv:2401.15884*.
- [12] Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2024). RAGAs: Automated Evaluation of Retrieval Augmented Generation. *EACL 2024 (System Demonstrations)*.
- [13] LangGraph documentation, <https://langchain-ai.github.io/langgraph/>